

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

Отчет по заданию №1
«Методы сортировки»
Вариант 3 / 4 / 1+4

Выполнил:
студент 103 группы
Казначеев А. С.

Преподаватель:
Малышев Н. Е.

Москва
2026

Содержание

1. Постановка задачи.....	2
2. Результаты экспериментов.....	3
3. Структура программы и спецификация функций.....	7
4. Отладка программы, тестирование функций.....	10
5. Анализ допущенных ошибок.....	11
Список литературы.....	12

1. Постановка задачи

В рамках задания требовалось реализовать на языке Си два метода сортировки массива вещественных чисел двойной точности (double) и провести их экспериментальное сравнение. В соответствии с вариантом 3 / 4 / 1 + 4:

- тип элементов массива: вещественные числа двойной точности (double)
- вид сортировки: упорядочивание по невозрастанию модулей, то есть при сравнении элементов знак числа не учитывается
- методы сортировки: метод «пузырька» и быстрая сортировка (рекурсивная реализация).

Дополнительные требования:

- Программа должна считывать со стандартного входа единственное число n -- длину массива, динамически выделять память под массивы (требовалось рассмотреть массивы длины 10, 100, 1000, 10000)
- Генерировать четыре исходных массива (уже упорядоченный по невозрастанию модулей, упорядоченный в обратном порядке и два массива со случайной расстановкой элементов)
- Запускать на каждом из них оба метода сортировки, подсчитывая число сравнений и число перемещений (обменов)
- Проверять корректность результата каждой сортировки с помощью автоматической тестирующей функции, не зависящей от конкретной реализации сортировки.

В случае, если входной параметр n прочесть не удалось, программа должна вывести сообщение об ошибке и завершить выполнение с ненулевым кодом возврата.

2. Результаты экспериментов

2.1. Методика эксперимента

Эксперименты проводились для длин массивов $n = 10, 100, 1000, 10000$. Для каждой длины генерировались четыре массива:

1. элементы уже упорядочены по невозрастанию модулей
2. элементы упорядочены по возрастанию модулей (обратный порядок)
3. случайная расстановка
4. случайная расстановка

Для каждого массива фиксировались количество сравнений и количество перемещений (обменов), выполненных каждым методом

2.2. Теоретические оценки

Для сортировки методом «пузырька» в литературе ([1], гл. 2, стр. 58) приводятся следующие оценки:

- число сравнений всегда равно $n(n-1)/2$;
- число обменов в лучшем случае (уже отсортированный массив) = 0, в среднем $\approx n(n-1)/4$, в худшем = $n(n-1)/2$

(Никлаус Вирт считал оценку для присваиваний, поэтому в его книге можно встретить множитель 3).

Для быстрой сортировки (рекурсивная реализация с выбором среднего элемента в качестве опорного) ([1], гл. 2, стр. 65-66)

- число сравнений в лучшем случае (всегда выбираем медиану в качестве границы) = $n \cdot \log(n)$;
число сравнений в среднем = $2 \cdot n \cdot \ln(n)$;
число сравнений в худшем случае = $O(n^2)$;
- число обменов в лучшем случае = $n \cdot \log(n)/6$;
в худшем случае (каждый раз в качестве опорного элемента выбирается наибольший/наименьший) = $O(n^2)$;

2.3. Таблицы результатов

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	45	45	45	45	45.00
100	4950	4950	4950	4950	4950.00
1000	499500	499500	499500	499500	499500.00
10000	49995000	49995000	49995000	49995000	49995000.00

Таблица 1. Количество сравнений при сортировке методом «пузырька»

Количество сравнений для метода «пузырька» не зависит от входных данных и в точности равно $n(n-1)/2$, что подтверждается результатами эксперимента.

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	0	45	22	21	22.00
100	0	4950	2252	2478	2420.00
1000	0	499474	252505	238812	247697.75
10000	0	49992527	25005317	25032959	25007700.75

Таблица 2. Количество перемещений при сортировке методом «пузырька»

На уже упорядоченном массиве (массив 1) обменов не происходит. На обратном упорядоченном массиве (массив 2) число обменов максимально и близко к числу сравнений (могут совпадать числа). На случайных массивах (массивы 3 и 4) число обменов составляет примерно половину от максимального, что согласуется с теоретической оценкой

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	31	34	44	38	36.75
100	606	610	880	862	739.50
1000	9067	9086	13132	13581	11216.50
10000	128141	128306	176526	189186	155539.75

Таблица 3. Количество сравнений при быстрой сортировке

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	6	11	8	11	9.00
100	63	112	177	181	133.25
1000	533	1033	2609	2617	1698.00
10000	7091	12114	34131	33899	21808.75

Таблица 4. Количество перемещений при быстрой сортировке

2.4. Выводы по экспериментам

1. Количество сравнений для метода «пузырька» не зависит от исходных данных и растёт квадратично(однако в точности не равно теоретическим расчетам, т.к. возможны равные элементы, для которых не происходит перемещений).

2. Быстрая сортировка существенно превосходит метод «пузырька» по числу сравнений и перемещений на больших n .

3. На упорядоченных данных (массивы 1 и 2) быстрая сортировка благодаря выбору среднего элемента в качестве опорного демонстрирует поведение, близкое к среднему случаю $O(n \log n)$, количество сравнений для массивов 1 и 2 близки.

4. На случайных массивах (3 и 4) число сравнений быстрой сортировки немного выше, чем на упорядоченных, что объясняется менее равномерным разбиением на каждом шаге рекурсии.

3. Структура программы и спецификация функций

Программа состоит из одного файла `sort.c`, содержащего функцию `main` и набор вспомогательных функций. Все функции, кроме `main`, объявлены со спецификатором `static`.

3.1. Функция `compar`

```
static int compar(double x, double y);
```

Функция сравнения элементов массива. Возвращает 1, если $|x| > |y|$ (то есть x должно стоять раньше y при сортировке по невозрастанию модулей), и 0 в противном случае. Увеличивает счётчик `com` на 1. Все сравнения элементов в программе выполняются только через эту функцию.

3.2. Генераторы массивов

```
static int generate_sorted(double* mas, unsigned int length);
```

Заполняет массив случайными по модулю числами, вставляя каждое новое число на нужное место (по невозрастанию модулей).

```
static int generate_sorted_rev(double* mas, unsigned int length);
```

Делает то же самое, после чего переворачивает массив.

```
static int generate_rand(double* mas, unsigned int length);
```

Заполняет массив случайными числами без сортировки.

Все генераторы используют `rand()` и `srand()` из `<stdlib.h>`.

Генерация числа:

```
double f = (double)rand() / RAND_MAX;  
f = (2.0 * f - 1.0) * DBL_MAX;
```

3.3. Метод Пузырька

```
static void bubblesort(double* mas, unsigned int length);
```

Вход: массив `mas` длины `length`.

Выход: массив отсортирован по невозрастанию модулей.

Алгоритм: Внешний цикл по i от 0 до `length-1`. Внутренний цикл по j от `length-1` до $i+1$. На каждом шаге внутреннего цикла сравниваются `mas[j]` и `mas[j-1]`; если `compar(mas[j], mas[j-1])` истинно (то есть $|mas[j]| > |mas[j-1]|$), элементы меняются местами. К концу i -й итерации первые $i+1$ элементов массива стоят на своих местах.

3.4. Quick Sort (рекурсивная реализация)

```
static void sort(int L, int R, double* mas);  
static void q_sort_rec(double* mas, unsigned int length);
```

1. Инициализируются два указателя: $i = L$, $j = R$ (границы)
2. Опорный элемент выбирается как средний элемент подмассива:
 $a = mas[(L + R) / 2];$
3. Организуется цикл `do while` внутри которого:
 - Указатель i увеличивается, пока `compar(mas[i], a)` истинно, то есть пока $|mas[i]| > |a|$;
 - Указатель j уменьшается, пока `compar(a, mas[j])` истинно, то есть пока $|mas[j]| < |a|$;
 - Если $i \leq j$ элементы $mas[i]$, $mas[j]$ меняются местами, счетчик увеличивается на 1.
4. Цикл повторяется, пока $i \leq j$.
5. После прохода рекурсивно вызывается функция для левой ($[L, j]$) и правой ($[i, R]$) частей, при условии $L < j$ и $i < R$ соответственно.

3.5. Тестирующая функция

```
static int test(double* mas, double* mas_sort, unsigned int length);
```

Вход: исходный массив `mas`, массив после сортировки `mas_sort`, их длина.

Выход: 0, если `mas_sort` -- корректно отсортированная версия `mas`; иначе 1.

Функция не знает, каким методом сортировали; проверка происходит в два этапа:

1. Проверяем упорядоченность: для всех i значение `compar(mas_sort[i+1], mas_sort[i])` ложно, то есть модули не возрастают.

2. Проверка, что `mas_sort` содержит те же элементы, что и `mas`, с теми же кратностями. Реализовано удалением из `mas_sort` каждого найденного элемента `mas`, после чего длина уменьшается. Если не нашли элемент из `mas` в `mas_sort`, то возвращаем 1.

3.6. Функция вывода результата

```
static void print_res(unsigned int length,  
                      int* mas, char* sort_type, char* param);
```

`length` — длина массива;

`mas` — указатель на массив (из 4 элементов), содержащий значения количеств перемещений/сравнений;

`sort_type` – метод сортировки;

`param` – параметр вывода(перемещения или сравнения).

Печатает одну таблицу результатов: заголовок с названием метода и параметра, строку со значениями для четырёх массивов и их средним.

3.7. Функция `main`

1. Считывает n через `scanf("%u", &length)`.

2. При ошибке чтения выводит сообщение и завершает работу с кодом возврата 1(при $n = 0$ выводит сообщение и завершает работу).

3. Выделяет память два массива длины n .

4. В цикле по четырём генераторам заполняет массив, копирует его, запускает обе сортировки, сохраняет `com/trans` в массивах `com_mas` и `trans_mas`, вызывает `test` и сохраняет его вердикт.

5. Выводит четыре таблицы и вердикты тестов.

4. Отладка программы, тестирование функций

1. Тестирование генераторов. Для малых n (1, 2, 3, 5) массив печатался на экран и проверялся визуально: упорядоченность, обратный порядок, случайность.

2. Тестирование сортировок. Каждая сортировка запускалась на массивах малой длины, результат выводился и сверялся с ожидаемым.

3. Автоматическое тестирование. Функция `test`, которая не зависит от алгоритма и проверяет корректность результата для любых n и любых входных данных. Подробное описание реализации функции `test` смотри в разделе 3.5

4. Сверка с теорией. Программа запускалась для $n = 10, 100, 1000, 10000$ на четырёх типах массивов; вердикт `test` для каждого запуска должен быть «ОК», также результаты работы программы сверялись с теоретическими оценками.

5. Проверка краевых случаев. $n = 1$, $n = 0$ (нулевая длина — сообщение и выход), n с нечисловым вводом.

5. Анализ допущенных ошибок

1. Ошибка с номером варианта. Сделал не свой вариант, из-за чего в программе присутствует реализация алгоритма Шелла.

2. Ошибка с подсчетом количества обменов. Если подсчитывать количество перемещений в функции `comp`, то функция `test` при использовании компаратора будет менять значения `com` и `trans`. Это приводило к неправильным расчетам (необходимо обнулять значения после работы тестирующей функции).

Большинство ошибок логические (тестирование, генерация), были ошибки с генерацией псевдослучайных чисел типа `double`.

Список литературы

[1] Wirt N. Algorithms and Data Structures. Oberon version: August 2004.